

INTERNAL CEESD REPORT

MIRGE-Com Performance Testing

M. Campbell

January 24, 2025

This report describes an effort to evaluate and model the computational performance of the Discontinuous Galerkin application *MIRGE-Com*.

1 Overview

This is a working document used in a project to develop an Advection-Diffusion Transport (ADT) operator in *MIRGE-Com*. We will develop and evaluate a performance model for this operator, and evaluate the implementation against a few other codes. Some possible othe codes:

- *DealII*
- *paranumal*
- *MFEM*
- *Nek5000*

2 Model Problem

The conservation system used for this performance study is the scalar (ADT). This system is chosen because it features the major mathematical and computational components required of many systems of interest, including the Navier-Stokes equations. The ADT system admits an analytic solution, and is relatively easily modeled computationally.

The equations of ADT will be described in the following sections, along with an analytic solution. In the following, the scalar field $Q(\mathbf{x}, t)$ is transported through a medium with advective velocity $\mathbf{v}$ and diffusivity D ; both are static, uniform parameters.

2.1 Governing Equations

In general, a scalar conservation law states that the time rate-of-change in the scalar field in a volume must be balanced by the “flow” of the scalar “material” across the volume boundary:

$$\frac{\partial Q}{\partial t} + \nabla \cdot \mathbf{F} = 0 \quad (1)$$

where $\mathbf{F}$ represents the total flux of the material. In the ADT system, the flux $\mathbf{F}$ consists of the advective and diffusive parts, $\mathbf{F}_{\text{adv}}$ and $\mathbf{F}_{\text{diff}}$, respectively:

$$\mathbf{F} = \mathbf{F}_{\text{adv}} + \mathbf{F}_{\text{diff}}$$

The advective flux, $\mathbf{F}_{\text{adv}}$, due to the motion of the medium at fixed, uniform velocity $\mathbf{v}$, is given by:

$$\mathbf{F}_{\text{adv}} = Q\mathbf{v}$$

The diffusive flux, $\mathbf{F}_{\text{diff}}$, is given by Fick’s law:

$$\mathbf{F}_{\text{diff}} = -D\nabla Q$$

Substituting the expressions for the fluxes into Equation 1 yields the following expression for the ADT system:

$$\frac{\partial Q}{\partial t} + \nabla \cdot (Q\mathbf{v} - D\nabla Q) = 0 \quad (2)$$

This represents the conservation of Q , where the change in Q within a volume is balanced by the advective and diffusive fluxes across its boundary.

2.2 Particular ADT Solution: Decaying Trig Function

Consider a decaying scalar wave field of the form:

$$Q = Ae^{-\alpha t} \cos(\mathbf{k} \cdot (\mathbf{x} - \mathbf{v}t)) \quad (3)$$

where A , α are constants, $\mathbf{v}$ is the advection velocity, and $\mathbf{k}$ is the wave vector which specifies the orientation of the wave.

2.2.1 Time derivative

The time derivative of Q :

$$\frac{\partial Q}{\partial t} = -\alpha Q + (\mathbf{k} \cdot \mathbf{v}) A e^{-\alpha t} \sin(\mathbf{k} \cdot (\mathbf{x} - \mathbf{v}t))$$

Making the following substitution:

$$\omega = \mathbf{k} \cdot \mathbf{v}, \quad \tilde{Q} = A e^{-\alpha t} \sin((\mathbf{k} \cdot \mathbf{x}) - \omega t) \quad (4)$$

the time derivative can be written more succinctly as:

$$\frac{\partial Q}{\partial t} = (\omega \tilde{Q} - \alpha Q) \quad (5)$$

2.2.2 Spatial derivatives

The gradient:

$$\nabla Q = -\mathbf{k} \tilde{Q} \quad (6)$$

The Laplacian:

$$\nabla^2 Q = \nabla \cdot -\mathbf{k} \tilde{Q} = -|\mathbf{k}|^2 Q \quad (7)$$

2.2.3 Condition for balance

Distributing the derivative operator in the ADT Equation 2 gives:

$$\frac{\partial Q}{\partial t} + \nabla Q \cdot \mathbf{v} - D \nabla^2 Q = 0$$

Substituting the derivative expressions from Equations 5, 6, and 7 yields:

$$(\omega \tilde{Q} - \alpha Q) - \omega \tilde{Q} + D |\mathbf{k}|^2 Q = 0$$

Gathering terms:

$$(D |\mathbf{k}|^2 - \alpha) Q = 0 \quad (8)$$

The condition for balance is:

$$\alpha = D |\mathbf{k}|^2 \quad (9)$$

The condition for balance is to choose the rate of decay α according to Equation 9.

2.2.4 Final expression

Finally, the exact analytic solution to the ADT system can be expressed as:

$$Q(\mathbf{x}, t) = A e^{-D |\mathbf{k}|^2 t} \cos(\mathbf{k} \cdot (\mathbf{x} - \mathbf{v}t)) \quad (10)$$

where A is an arbitrary constant. The main parameters setting the solution character are the diffusivity D , and the wave vector $\mathbf{k}$, and the character of the medium is set by velocity $\mathbf{v}$. Typically, one would set the wavelength of the wave, λ , and a unit vector $\hat{\mathbf{k}}$ specifying the orientation of the wave. Then the wave vector is set as:

$$\mathbf{k} = \frac{2\pi}{\lambda} \hat{\mathbf{k}} \quad (11)$$

3 DG Discretization

MIRGE-Com currently employs a DG strategy akin to the *BR1* algorithm outlined in [Bassi_1997], but with thermal terms and chemical reaction sources as outlined in [Ihme_2014] and [Cook_2009].

3.1 Recasting ADT for DG

The ADT equations are rewritten as the following coupled system for two unknowns, $\mathbf{Q}$, and $\mathbf{\Sigma}$.

$$\mathbf{\Sigma} - \nabla Q = \mathbf{0} \quad \text{auxiliary eqn} \quad (12)$$

$$\frac{\partial Q}{\partial t} + \underbrace{\nabla \cdot \mathbf{F}_{\text{adv}}(Q) - \nabla \cdot \mathbf{F}_{\text{diff}}(Q, \mathbf{\Sigma})}_{=\nabla \cdot \mathbf{F}(Q, \mathbf{\Sigma})} = 0 \quad \text{primary eqn} \quad (13)$$

At each time level, we first solve the auxiliary equation for the gradient of Q , $\mathbf{\Sigma}$, which is used to compute the diffusive fluxes $\mathbf{F}_{\text{diff}}$. Then we solve the primary equation for the time rate of change of Q .

Let Ω_h denote a collection of disjoint elements E . The DG method constructs approximations Q_h , and $\mathbf{\Sigma}_h$, to Q , and $\mathbf{\Sigma}$ respectively, in discontinuous finite element spaces. Here, and in what follows, we use the subscript h to denote discrete quantities. For any integer k , we define the following discontinuous spaces of piecewise (vector-valued) polynomial functions:

$$\mathbf{V}_h^k = \left\{ \mathbf{v} \in L^2(\Omega_h)^N : \mathbf{v}|_E \in [P^k(E)]^N, \forall E \in \Omega_h \right\}, \quad (14)$$

$$\mathbf{W}_h^k = \left\{ \mathbf{w} \in L^2(\Omega_h)^{N \times d} : \mathbf{w}|_E \in [P^k(E)]^{N \times d}, \forall E \in \Omega_h \right\}, \quad (15)$$

where the number of equations $N = 1$, and d is the spatial dimension. Here, $P^k(E)$ denotes a polynomial space on E consisting of functions of degree $\leq k$. The DG formulation is obtained by multiplying by “test functions” $\mathbf{v}_h \in \mathbf{V}_h^k$, and $\mathbf{w}_h \in \mathbf{W}_h^k$ (one for each equation respectively) and integrating over each element.

The resulting DG problem reads as follows: find $(Q_h, \mathbf{\Sigma}_h) \in \mathbf{V}_h^k \times \mathbf{W}_h^k$ such that $\forall (\mathbf{v}_h, \mathbf{w}_h) \in \mathbf{V}_h^k \times \mathbf{W}_h^k$, we have:

$$\sum_{E \in \Omega_h} \left[\int_E \mathbf{v}_h \frac{\partial Q_h}{\partial t} d\Omega + \oint_{\partial E} \mathbf{v}_h h d\sigma - \int_E \nabla \mathbf{v}_h \cdot \mathbf{F}(Q_h, \mathbf{\Sigma}_h) d\Omega \right] = 0 \quad (16)$$

$$\sum_{E \in \Omega_h} \left[\int_E \mathbf{w}_h \mathbf{\Sigma}_h d\Omega - \oint_{\partial E} \mathbf{w}_h \mathbf{H}_s d\sigma + \int_E \nabla \mathbf{w}_h Q_h d\Omega \right] = 0, \quad (17)$$

where $h = h_a - h_d$ is a composite *numerical flux* respectively representing the advective and diffusive contributions to the physical flux $(\mathbf{F}_{\text{adv}}(Q_h) - \mathbf{F}_{\text{diff}}(Q_h, \mathbf{\Sigma}_h) \cdot \mathbf{n})$, with $\mathbf{n}$ being the outward facing unit normal with respect to the local element. $\mathbf{H}_s$ is the *gradient numerical flux* ($Q_h \mathbf{n}$) for the auxiliary equation. More details on the numerical fluxes are provided in the next section.

Since $\mathbf{F}_{\text{adv}}(Q_h) \cdot \mathbf{n}$ is discontinuous, the quantities are allowed to vary on either side of a shared element boundary. That is, $\mathbf{F}_{\text{adv}}(Q_h^+) \cdot \mathbf{n}^+ \neq \mathbf{F}_{\text{adv}}(Q_h^-) \cdot \mathbf{n}^-$. Here, $\mathbf{n}^+$ and $\mathbf{n}^-$ denote the “cell-local” (outward pointing) unit normals for elements E^+ and E^- (respectively) which share a face: $\partial E^+ \cap \partial E^- \neq \emptyset$. Similarly for the diffusive and auxiliary fluxes $\mathbf{F}_{\text{diff}}(Q_h, \mathbf{\Sigma}_h) \cdot \mathbf{n}$, and $Q_h \mathbf{n}$.

Expanding out the trial and test functions (component-wise) in terms of the local element basis in each element:

$$Q(\mathbf{x}, t)_h|_E = \sum_{i=1}^n Q_i(t) \phi_i^k(\mathbf{x}), \quad \Sigma(\mathbf{x})_h|_E = \sum_{i=1}^n \Sigma_i \phi_i^k(\mathbf{x}), \quad (18)$$

$$\mathbf{v}(\mathbf{x})_h|_E = \sum_{i=1}^n \mathbf{v}_i \phi_i^k(\mathbf{x}), \quad \mathbf{w}(\mathbf{x})_h|_E = \sum_{i=1}^n \mathbf{w}_i \phi_i^k(\mathbf{x}), \quad (19)$$

allows us to obtain a set of algebraic equations for the conserved state Q_h and the auxiliary gradient variable Σ_h . That is, for each $j = 1, \dots, \dim P^k$, we have:

$$\begin{aligned} \frac{d}{dt} \sum_{E \in \Omega_h} \int_E \phi_j^k Q_h d\Omega &= \sum_{E \in \Omega_h} \left[\int_E \nabla \phi_j^k \cdot (\mathbf{F}_{\text{diff}}(Q_h, \Sigma_h) - \mathbf{F}_{\text{adv}}(Q_h)) d\Omega \right] \\ &\quad - \sum_{E \in \Omega_h} \left[\oint_{\partial E} \phi_j^k h_a(Q_h^\pm; \mathbf{n}) d\sigma + \oint_{\partial E} \phi_j^k h_d(Q_h^\pm, \Sigma_h^\pm; \mathbf{n}) d\sigma \right], \end{aligned} \quad (20)$$

$$\sum_{E \in \Omega_h} \int_E \phi_j^k \Sigma_h d\Omega = \sum_{E \in \Omega_h} \left[\oint_{\partial E} \phi_j^k \mathbf{H}_s(Q_h^\pm; \mathbf{n}) d\sigma - \int_E \nabla \phi_j^k Q_h d\Omega \right]. \quad (21)$$

$$(22)$$

With the aid of the DG element-local mass and stiffness operators are defined as:

$$\mathcal{M}_{ij} = \int_{E_h} \phi_i \phi_j dV, \quad \mathcal{S}_{ij} = \int_{E_h} \phi_i \nabla \phi_j dV, \quad \mathcal{M}_{ij}^f = \int_{\partial E_h} \hat{\phi}_i \hat{\phi}_j dS, \quad (23)$$

$$(24)$$

where $\hat{\phi}$ are the shape functions ϕ restricted to the element boundary ∂E_h , we can express the system as:

$$\sum_{E \in \Omega_h} \mathcal{M} \dot{Q}_h = \sum_{E \in \Omega_h} \left(\mathcal{S} \mathbf{F}(Q_h, \Sigma_h) - \mathcal{M}^f h(Q_h^\pm, \Sigma_h^\pm; \mathbf{n}) \right), \quad (25)$$

$$\sum_{E \in \Omega_h} \mathcal{M} \Sigma_h = \sum_{E \in \Omega_h} \left(\mathcal{M}^f \mathbf{H}_s(Q_h^\pm; \mathbf{n}) - \mathcal{S} Q_h \right), \quad (26)$$

and the element-local expression for the primary and auxilliary equations:

$$\dot{Q}_h = \mathcal{M}^{-1} \left(\mathcal{S} \mathbf{F} - \mathcal{M}^f h \right), \quad (27)$$

$$\Sigma_h = \mathcal{M}^{-1} \left(\mathcal{M}^f \mathbf{H}_s - \mathcal{S} Q_h \right) \quad (28)$$

The DG operators $\mathcal{M}$, $\mathcal{S}$, and $\mathcal{M}^f$ are provided by the *MIRGE-Com* infrastructure, so we must express the ADT-specific constructs Q , $\mathbf{F} = \mathbf{F}_{\text{diff}} - \mathbf{F}_{\text{adv}}$, h and $\mathbf{H}_s$ to advance the ADT system in *MIRGE-Com*.

3.2 Numerical fluxes

Numerical fluxes are responsible for calculating the fluxes at all *internal* DG element boundaries, and are also sometimes used to transmit boundary conditions into the domain at the domain boundary. Numerical fluxes must account for the discontinuities at element faces, and calculate a single valued flux that both elements agree on. That is, they must be functions of both $\pm$ states, and must produce a consistent flux.

For a conservation law $\frac{\partial Q}{\partial t} + \nabla \cdot \mathbf{F}(Q) = \mathbf{S}$, the numerical flux h must satisfy the consistency relations

$$h(Q^+, Q^-; \mathbf{n}) = \mathbf{F}(Q) \cdot \mathbf{n} \quad (29)$$

$$h(Q^+, Q^-; \mathbf{n}) = -h(Q^-, Q^+; -\mathbf{n}) \quad (30)$$

Specifically, for our system of equations, we use numerical flux functions h_a for the advective fluxes, h_d for the diffusive fluxes, and H_s for the gradient fluxes used in the auxiliary equation. More details on each of these numerical fluxes is provided in the following sections.

3.2.1 Numerical flux for advective terms

In this performance-oriented study, where we have fixed velocity $\mathbf{v}$ and transport only smooth $\mathcal{C}_\infty$ functions in Q , we choose simple central numerical fluxes for both the advective and diffusive terms.

The central numerical flux for the divergence of the advective transport flux is implemented as:

$$h_a(Q_h^+, Q_h^-; \mathbf{n}) = \frac{1}{2} (\mathbf{F}_{\text{adv}}(Q_h^+) + \mathbf{F}_{\text{adv}}(Q_h^-)) \cdot \mathbf{n}, \quad (31)$$

3.2.2 Numerical Fluxes for diffusive terms

The numerical flux function used for the divergence of the diffusive transport flux is implemented as follows:

$$h_d(Q_h^+, \Sigma_h^+, Q_h^-, \Sigma_h^-; \mathbf{n}) = \frac{1}{2} (\mathbf{F}_{\text{diff}}^+ + \mathbf{F}_{\text{diff}}^-) \cdot \mathbf{n}, \quad (32)$$

where $\mathbf{F}_{\text{diff}}^\pm \equiv \mathbf{F}_{\text{diff}}(Q_h^\pm, \Sigma_h^\pm)$, is the diffusive flux function computed for the $(\pm)$ sides of the element boundary, respectively.

3.2.3 Numerical fluxes for the auxiliary equations (gradients)

The numerical flux function used for the auxiliary equations (i.e. when computing the gradient of the fluid solution and temperature) is implemented as follows:

$$\mathbf{H}_s(Q_h^+, Q_h^-; \mathbf{n}) = \frac{1}{2} ((Q_h^+ + Q_h^-)) \mathbf{n} \quad (33)$$

4 Simulation Setup & Configuration

Configuration parameters:

- λ : solution wavelength
- $\hat{\mathbf{k}}$: solution orientation
- $\mathbf{v}$: advective velocity (arbitrary)
- N_{el} : number of elements per axis

The remaining parameters are set as follows:

- simulation timestep: $\Delta t = \frac{\lambda}{\sqrt{2}N_{\text{el}}|\mathbf{v}|}$

4.1 Domain

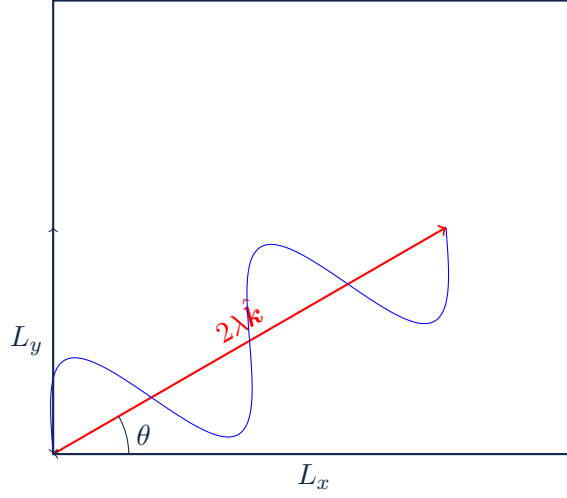


Figure 1: 2D domain example with wave orientation $\hat{\mathbf{k}}$, wavelength λ , and side lengths $[L_x, L_y] = [\frac{2\lambda}{\hat{\mathbf{k}} \cdot \hat{\mathbf{x}}}, \frac{\lambda}{\hat{\mathbf{k}} \cdot \hat{\mathbf{y}}}]$

4.2 Initialization

4.3 Simulation Parameters & Resource Configuration

4.3.1 Domain decomposition

4.3.2 Partition-to-processor parameters

5 Performance Model

5.1 Formulation

Write out the matrix/operator form of system equations in more detail, for use in the performance model.

$$\dot{Q} = \mathbb{M}^{-1} \left(\mathbb{S}_{\hat{x}} F_x + \mathbb{S}_{\hat{y}} F_y + \mathbb{S}_{\hat{z}} F_z + \sum_{\text{faces}} \mathbb{M}_f h \right) \quad (34)$$

$$\Sigma = \mathbb{M}^{-1} \left(\sum_{\text{faces}} \mathbb{M}_f Q_h \hat{\mathbf{n}} - \langle \mathbb{S}_{\hat{x}} Q, \mathbb{S}_{\hat{y}} Q, \mathbb{S}_{\hat{z}} Q \rangle \right) \quad (35)$$

5.2 Elements and DOF Counting

Explain how to calculate the sizes of the DG operators. Simplex elements (lines, triangles and tetrahedra, in 1, 2, and 3 dimensions, resp.) have $d + 1$ vertices, and faces per element, where d is the number of spatial dimensions. For example, a 3d simplex (i.e. a tetrahedron) has 4 vertices and 4 faces. Each face of a simplex is itself just a simplex of dimension $d - 1$.

The number of DOFs (or nodes, or modes) on the simplicial element is order-dependent. To represent a scalar field on a simplex element, the number of DOFs per element of a given order p is $d_v = \frac{\# \text{ volume dofs}}{\text{element}} = \frac{(p+d)!}{(p!)(d!)}$.

For example, a $p = 1$ tetrahedral element will have 4 nodes, and a scalar field represented on a tetrahedral elements will require $d_v = \frac{4!}{3!} = 4$ nodal DOFs. The tet also has 4 triangular faces, and each face has $d_f = \frac{3!}{2!} = 3$ DOFs, when we represent a scalar field on the face. Order $p = 2$ tets have 10 volume nodes, and each triangular face has 6 nodes.

Tensor product elements will have 2^d vertices, and $2d$ faces per element.

We denote the number of DOFs per element, and per face as d_v , and d_f , respectively. Likewise, in the following the number of elements will be denoted as N_v and the total number of faces by N_f , while the number of faces per element is denoted by n_f . The number of faces on any boundary b_i is denoted N_{b_i} . Note that $N_f = N_v n_f$.

5.3 *MIRGE-Com* DG Operators

The element-local operators pre-calculated and provided by *MIRGE-Com*, and their sizes are as follows:

- $\mathbb{M}^{-1}$: Inverse mass operator, $d_v \times d_v$
- $\mathbb{M}_f$: Face mass operator, $d_v \times d_f$
- $\mathbb{P}$: Projection operator $d_f \times d_v$
- $\mathbb{S}_j$: Stiffness operator $d_v \times d_v$

5.4 Periodic Boundaries

The current implementation assumes periodic boundaries in all dimensions, eliminating the need for domain-boundary-specific numerical flux handling. This simplifies computation and aligns with future extensions to more complex models.

5.5 Algorithm For ADT RHS

Write a pseudo-code explanation of every step, especially those steps that are expected to impact performance.

Algorithm 1 Project Q to Element Faces

```

1: function PROJECTQ( $Q_v$ )
2:   Input:  $Q_v$                                 ▷  $Q$  on the volume discretization
3:   Output:  $Q_f^\pm$                                 ▷  $Q$  tracepair on all faces
4:   for  $n = 1$  to  $N_v$  do
5:     for  $m = 1$  to  $n_f$  do
6:       Project  $Q_v$  to face:  $Q_f^\pm = \mathbb{P}(Q_v)$                                 ▷  $d_v \times d_f$ 
7:     end for
8:   end for
9:   for each partition boundary  $b_i$  do                                ▷ Exchange  $Q^\pm$  across all partition boundaries
10:    for  $n = 1$  to  $N_{b_i}$  do
11:      Send  $Q^-$                                 ▷ sends  $d_f$  values
12:      Recv  $Q^+$                                 ▷ receives  $d_f$  values
13:    end for
14:  end for
15:  Return:  $Q_f^\pm$ 
16: end function

```

Algorithm 2 Project ∇Q to Element Faces

```
1: function PROJECTGRADQ( $\nabla Q_v$ )
2:   Input:  $\nabla Q_v$  ▷  $\nabla Q$  on the volume discretization
3:   Output:  $\nabla Q_f^\pm$  ▷  $\nabla Q$  tracepair on all faces
4:   for  $n = 1$  to  $N_v$  do
5:     for  $m = 1$  to  $n_f$  do
6:       for  $i = 1$  to  $d$  do
7:          $(\nabla Q_f^\pm)_i = \mathbb{P}((\nabla Q_v)_i)$  ▷ projects  $i$ th component of  $\nabla Q_v$  to face
8:       end for
9:     end for
10:  end for
11:  for each partition boundary  $b_i$  do ▷ Exchange  $\nabla Q^\pm$  across all partition boundaries
12:    for  $n = 1$  to  $N_{b_i}$  do
13:      Send  $\nabla Q^-$  ▷ sends  $d \times d_f$  values
14:      Recv  $\nabla Q^+$  ▷ receives  $d \times d_f$  values
15:    end for
16:  end for
17:  Return:  $\nabla Q_f^\pm$ 
18: end function
```

Algorithm 3 Compute fluxes for advection

```
1: function COMPUTEADVECTIONFLUXES( $Q_v, Q_f^\pm, \mathbf{v}$ )
2:   Input:  $Q_v$  ▷  $Q$  on the volume discretization
3:   Input:  $Q_f^\pm$  ▷  $Q$  tracepair on all faces
4:   Input:  $\mathbf{v}$  ▷  $d$ -vector advective velocity
5:   Output:  $\mathbf{F}_{\text{adv}}$  ▷ Advective flux on volume discretization
6:   Output:  $h_f$  ▷ Numerical flux for advection on all faces
7:   for  $n = 1$  to  $N_v$  do
8:     Compute advection flux:  $\mathbf{F}_{\text{adv}} = Q_v \mathbf{v}$ 
9:   end for
10:  for  $n = 1$  to  $N_f$  do
11:    Compute numerical flux:  $h_f = \{\{Q_f^\pm \mathbf{v}\}\} \cdot \hat{\mathbf{n}}$ 
12:  end for
13:  Return:  $\mathbf{F}_{\text{adv}}, h_f$ 
14: end function
```

Algorithm 4 Compute flux terms for solution gradient

```
1: function COMPUTEGRAIDENTFLUXES( $Q_v, Q_f^\pm$ )
2:   Input:  $Q_v$  ▷  $Q$  on the volume discretization
3:   Input:  $Q_f^\pm$  ▷  $Q$  tracepair on all faces
4:   Output:  $\delta Q_v$  ▷ Volume term for gradient
5:   Output:  $S_f$  ▷ Numerical flux for gradient on all faces
6:   for  $n = 1$  to  $N_v$  do
7:     Compute gradient volume term:  $\delta Q_v = \langle S_x Q_v, S_y Q_v, S_z Q_v \rangle$ 
8:   end for
9:   for  $n = 1$  to  $N_f$  do
10:    Compute gradient numerical flux:  $S_f = \{\{Q_f^\pm\}\} \hat{n}$ 
11:  end for
12:  Return:  $\delta Q_v, S_f$ 
13: end function
```

Algorithm 5 Compute solution gradient

```
1: function COMPUTEGRAIDENT( $Q_v, Q_f^\pm$ )
2:   Input:  $Q_v$  ▷  $Q$  on the volume discretization
3:   Input:  $Q_f^\pm$  ▷  $Q$  tracepair for all faces
4:   Output:  $\nabla Q_v$  ▷  $\nabla Q$  on the volume discretization
5:   Output:  $\nabla Q_f^\pm$  ▷  $\nabla Q$  tracepair on all faces
6:    $\delta Q_v, S_f = \text{COMPUTEGRAIDENTFLUXES}(Q_v, Q_f^\pm)$ 
7:   for  $n = 1$  to  $N_v$  do
8:     for  $i = 1$  to  $d$  do
9:        $(\nabla Q_v)_i = \mathbb{M}^{-1}((\delta Q_v)_i + \mathbb{M}_f((S_f)_i))$  ▷ computes  $i$ th component of  $\nabla Q_v$ 
10:    end for
11:  end for
12:   $\nabla Q_f^\pm = \text{PROJECTGRADQ}(\nabla Q_v)$ 
13:  Return:  $\nabla Q_v, \nabla Q_f^\pm$ 
14: end function
```

Algorithm 6 Compute fluxes for diffusion

```
1: function COMPUTEDIFFUSIONFLUXES( $\nabla Q_v, \nabla Q_f^\pm, D$ )
2:   Input:  $\nabla Q_v$  ▷  $\nabla Q$  on the volume discretization
3:   Input:  $\nabla Q_f^\pm$  ▷  $\nabla Q$  tracepair on all faces
4:   Output:  $F_{\text{diff}}$  ▷ Diffusion flux on the volume discretization
5:   Output:  $H_f$  ▷ Numerical flux for diffusion on all faces
6:   for  $n = 1$  to  $N_v$  do
7:      $F_{\text{diff}} = -D \nabla Q_v$ 
8:   end for
9:   for  $n = 1$  to  $N_f$  do
10:    Compute numerical flux  $H_f = \{\{-D \nabla Q_f^\pm\}\} \cdot \hat{n}$ 
11:  end for
12:  Return:  $F_{\text{diff}}, H_f$ 
13: end function
```

Algorithm 7 Apply Inverse Mass Operator for $\dot{Q}$

```
1: function APPLYINVERSEMASSDIV( $\mathbf{F}_v, h_f$ )
2:   Input:  $\mathbf{F}_v$  ▷ Total physical flux on volume discretization
3:   Input:  $h$  ▷ Total numerical flux on all faces
4:   Output:  $\dot{Q}_v$  ▷ Time rate of change for  $Q$  on volume discretization
5:   for  $n = 1$  to  $N_v$  do
6:      $\nabla \cdot \mathbf{F} = \mathbb{S}_{\hat{x}} \mathbf{F}_x + \mathbb{S}_{\hat{y}} \mathbf{F}_y + \mathbb{S}_{\hat{z}} \mathbf{F}_z$ 
7:      $\dot{Q}_v = \mathbb{M}^{-1}(\mathbb{M}_f(h) + \nabla \cdot \mathbf{F})$ 
8:   end for
9: end function
```

Algorithm 8 Compute RHS ($\dot{Q}$) for ADT System

```
1: function ADVECTIONDIFFUSIONRHS( $Q_v, \mathbf{v}, D$ )
2:   Input:  $Q_v$  ▷ Solution  $Q$  on the volume discretization
3:   Input:  $\mathbf{v}$  ▷  $d$ -vector velocity (parameter)
4:   Input:  $D$  ▷ Diffusivity (parameter)
5:   Output:  $\dot{Q}_v$  ▷ Time rate of change for  $Q$  on volume discretization
6:    $\mathbf{F}_v = (\mathbf{0})_v$  ▷ Initialize to zeros
7:    $h_f = (0)_f$  ▷ Initialize to zeros
8:   if Advection Enabled or Diffusion Enabled then
9:      $Q_f^\pm = \text{PROJECTQ}(Q_v)$ 
10:  end if
11:  if Advection Enabled then
12:     $\mathbf{F}_v, h_f = \text{COMPUTEADVECTIONFLUXES}(Q_v, Q_f^\pm, \mathbf{v})$ 
13:  end if
14:  if Diffusion Enabled then
15:     $\nabla Q_v, \nabla Q_f^\pm = \text{COMPUTEGRADQ}(Q_v, Q_f^\pm)$ 
16:     $\mathbf{F}_v, h_f = \mathbf{F}_v, h_f + \text{COMPUTEDIFFUSIONFLUXES}(\nabla Q_f, \nabla Q_f^\pm, D)$ 
17:  end if
18:   $\dot{Q}_v = \text{APPLYINVERSEMASSDIV}(\mathbf{F}_v, h_f)$ 
19:  Return:  $\dot{Q}_v$ 
20: end function
```

5.6 Expected performance

Based on the previous two sections, write out the expected performance of each component. Make a count of the components and for each, include:

- Number of FLOPs
- Memory Load-and-Stores

5.6.1 General operators

What FLOPs, memory requirements do we expect for the operations appearing in the DG code?

A matrix-matrix (MatMat) multiplication of matrices with sizes described by: $(m \times n) \times (n \times k)$ should, at least naively, involve $m k (2n - 1)$ FLOPS, with $mn + nk + mk$ LoadStores, not counting storing/accessing intermediate values. For a MatVec, $k = 1$. Scalar operations may simply be counted.

5.6.2 Algorithm 1: Solution Projection

Each projection of Q_v to Q_f for each face should take $d_v(2d_f - 1)$ FLOPs with $d_v d_f + d_f + d_v$ LoadStores. There is one projection for every face in the mesh (i.e. every face of every element) which gives $N_v n_f$ projections. Neglecting communication, Algorithm 1 should perform as:

- FLOPs: $N_v n_f d_v (2d_f - 1)$
- LoadStores: $N_v n_f (d_v d_f + d_f + d_v)$

5.6.3 Algorithm 2: Gradient Projection

Each projection of each component of ∇Q_v to the component of ∇Q_f for each face should take $d_v(2d_f - 1)$ FLOPs with $(d_v d_f + d_f + d_v)$ LoadStores. There is one projection for every face in the mesh (i.e. every face of every element) for each component which gives $d N_v n_f$ projections. Neglecting communication, Algorithm 2 should perform as:

- FLOPs: $d N_v n_f d_v (2d_f - 1)$
- LoadStores: $d N_v n_f (d_v d_f + d_f + d_v)$

5.6.4 Algorithm 3: Advection Flux Computation

Computing the physical flux for each element gives $d N_v d_v$ total FLOPs. The solution must be loaded and the vector components of the flux stored, which gives $(N_v + d N_v) d_v$ LoadStores. Nice, this gives the advective flux computation a fixed computational intensity (CI) of $\frac{d}{d+1}$. The physical flux computation should perform as:

- FLOPs: $d N_v d_v$
- LoadStores: $N_v (d + 1) d_v$

Computing the numerical flux for each face in the mesh requires loading Q_f^- , Q_f^+ , and the d components of $\hat{n}$, and storing the result. 2 FLOPs compute the solution average and $(2d - 1)$ FLOPs for the dot product. This part of the algorithm has a fixed CI of $\frac{(2d+1)}{d+3}$. This is done for each face in the mesh, so computing the advection numerical flux should perform as:

- FLOPs: $N_v n_f (2d + 1) d_f$
- LoadStores: $N_v n_f (d + 3) d_f$

5.6.5 Algorithm 7: Apply Inverse Mass Operator

First, the divergence of the flux is computed which requires loading dd_v^2 for the $\mathbb{S}$ operators and dd_v components of the flux, and then storing the d_v result. The FLOPs are d times scalar volume operators ($d_v(2d_v - 1)$) FLOPs and $(d - 1)d_v$ FLOPs for the dot product. The divergence operation should perform:

- FLOPs: $N_v d_v (2dd_v + 1)$
- LoadStores: $N_v d_v (d(d_v + 1) + 1)$

The application of the face mass operator to the numerical flux requires loading $d_v d_f$ for the operator and d_f values of the numerical flux for each face, and storing the d_v results for each element. $d_v(2d_f - 1)$ FLOPs per face, and $n_f - 1$ FLOPs are required to accumulate the result into the volume data structure for each element.

- FLOPs: $N_v(n_f d_v 2d_f - 1)$
- LoadStores: $N_v(n_f d_f (d_v + 1) + d_v)$

The formation of the operand for the inverse face mass operator requires d_v FLOPs per element. The operator FLOPs are $d_v(2d_v - 1)$. The LoadStores are $d_v(d_v + 2)$.

- FLOPs: $N_v 2d_v^2$
- LoadStores: $N_v d_v (d_v + 2)$

5.7 Additional considerations

- Kernel content and number of kernels in *MIRGE-Com* is not the same as those “on paper”
- With GPU calculations, there will be additional costs and overheads of data transfer, and queueing (floor)
- *MIRGE-Com* applies grid metrics on-the-fly as near as I can tell, will change the observed performance

6 Parallel Model Considerations

Additional considerations for extending the performance model for distributed computation.

6.1 Partition Boundaries and MPI Communication

Each MPI rank processes a *partition* of the domain, with communication required to exchange the solution and its gradient across partition boundaries. Key considerations include:

- **Communication Volume:** For scalar Q , one message per face is sent for Q , and d messages are sent for ∇Q . For CNS extensions, $N_{\text{equations}}$ components multiply this communication volume. (i.e. there is one message per *MIRGE-Com* DOFArray)
- **Data Transfer:** Communication occurs on the host. Data must be transferred between the GPU and host before and after MPI exchange, introducing latency and bandwidth costs.
- **Partition Load Balancing:** Domain partitioning with Metis ensures approximate load balance, though face counts and neighbor counts (N_p) can vary across ranks.

6.2 Communication Overheads

The MPI communication overhead can be modeled as:

$$T_{\text{comm}} = N_p \cdot N_f \cdot d_f \cdot (T_{\text{send}} + T_{\text{recv}}) + 2T_{\text{transfer}},$$

where:

- N_p : Number of neighbors per rank,
- N_f : Number of partition-boundary faces,
- $T_{\text{send}}, T_{\text{recv}}$: Costs for sending and receiving messages,
- T_{transfer} : Cost of transferring data between GPU and host.

6.3 Rank-Level Variations and Bottlenecks

The number of neighbors (N_p) and partition-boundary faces varies significantly across ranks:

- **Neighbor Count Variability:** Even in simple 1D partitioning, end ranks may have only one neighbor, while others have two. In higher-dimensional partitioning with Metis, N_p can vary widely, reflecting the unstructured nature of the mesh.
- **Partition-Boundary Face Counts:** The number of faces on a partition boundary differs across ranks but is symmetric for the two ranks sharing the boundary.

To model parallel performance, a rank-by-rank analysis is essential, as overall application performance will depend on the slowest rank.

6.4 Communication and Computation Overlap

Currently, no overlap exists between computation and communication within a given rank. This introduces potential inefficiencies, as ranks must wait for communication to complete before resuming computation. Future work will explore strategies to mitigate this bottleneck.

6.5 Future Work

Stubs for incorporating communication costs into the performance model are prepared. The next steps include measuring transfer and MPI latency costs on target platforms.

7 Numerical Results

7.1 ADT

7.2 Performance